

Routage et gestion des URL dans les applications web monopages

Dans les applications web monopages (Single Page Applications - SPA), la gestion des URL et du routage est essentielle pour offrir une expérience utilisateur fluide et cohérente. Contrairement aux applications web traditionnelles où chaque URL correspond à une page distincte servie par le serveur, les SPA utilisent le routage côté client pour gérer la navigation entre différentes vues ou composants sans recharger la page entière.

Étapes pour gérer les URL dans une SPA

Capter l'url courante via JavaScript

Pour capter l'URL courante dans une application monopage, on peut utiliser l'objet `window.location`. Cet objet fournit des informations sur l'URL actuelle, y compris le chemin, les paramètres de requête et le fragment (hash).

On peut également utiliser l'interface `URL` pour une manipulation plus facile des différentes parties de l'URL.

- `pathname` : Le chemin de l'URL (ex: `/page1`)
- `search` : Les paramètres de requête (ex: `?id=123&categorie=livres`)
- `hash` : Le fragment de l'URL (ex: `#section1`) si nécessaire. On ne s'en servira pas pour le routage principal mais peut être utile pour des ancres ou sections spécifiques.

```
const url = new URL(window.location.href);
console.log(url.pathname); // Affiche le chemin de l'URL EX: /page1
console.log(url.search); // Affiche les paramètres de requête. EX: ?
id=123&categorie=livres
console.log(url.hash); // Affiche le fragment (hash) EX: #section1
```

Événements liés à l'historique du navigateur

L'API History du navigateur permet de manipuler l'historique de navigation sans recharger la page. Les événements `popstate` sont déclenchés lorsque l'utilisateur navigue en arrière ou en avant dans l'historique.

```
class Routeur {
  constructor() {
    window.addEventListener("popstate",
    this.onChangeementHistorique.bind(this));
  }

  onChangeementHistorique(event) {
    // Gérer le changement d'état ici
    const url = new URL(window.location.href);
    naviguerVers(url.pathname, url.search, url.hash); // Fonction
    personnalisée pour gérer la navigation
  }
}
```

```
}  
}
```

Événements liés aux clics sur les liens

Pour gérer la navigation via des liens, il est courant d'empêcher le comportement par défaut des liens HTML et de gérer la navigation via JavaScript. Pour simplifier, on place un écouteur d'événements sur le conteneur principal qui englobe tous les liens de navigation et utilise le bubbling des événements pour capturer les clics sur les liens.

```
class Routeur {  
  constructor() {  
    document.body.addEventListener("click", this.onClicLien.bind(this));  
  }  
  
  onClicLien(evenement) {  
    if (evenement.closest("[data-lien]")) {  
      // Empêcher le rechargement de la page  
      evenement.preventDefault();  
  
      const url = new URL(evenement.target.href);  
      naviguerVers(url.pathname, url.search, url.hash); // Fonction  
      personnalisée pour gérer la navigation  
    }  
  }  
}
```

Créer un objet pour cartographier les routes

Dans la classe de routage, on peut créer un objet qui mappe les chemins d'URL aux fonctions ou composants correspondants. La clé est le chemin d'URL, et la valeur est le nom de la fonction ou du composant à charger. (Voir exemple complet à la fin)

```
import AccueilVue from "./views/AccueilVue.js";  
import PageProduitVue from "./views/PageProduitVue.js";  
import PageProduitDetailVue from "./views/PageProduitDetailVue.js";  
  
class Routeur {  
  constructor() {  
    this.routes = {  
      "/": AccueilVue,  
      "/produits": PageProduitVue,  
      "/produits/:id": PageProduitDetailVue,  
      // Ajouter d'autres routes ici  
    };  
  }  
}
```

Gérer le routage dynamique avec des paramètres

Lorsqu'il faut mettre la jour la vue en fonction des paramètres dynamiques dans l'URL (comme un ID de produit), on peut utiliser des expressions régulières pour faire correspondre les routes et extraire les paramètres.

On récupère l'url courante, on transforme la chaîne en tableau en découpant par les `/`, puis on compare chaque segment avec les routes définies dans l'objet de routage.

Ensuite, on passe la classe de la vue dans une variable et on l'instancie avec les paramètres dynamiques si nécessaire.

```
miseAJour() {
  const url = new URL(window.location.href);
  const cheminActuel = url.pathname;
  const paramRecherche = url.search;
  const paramHash = url.hash;

  const segmentsActuels = cheminActuel.split("/");
  const routeActuelle = segmentsActuels[0] === "" ? "/" : segmentsActuels[0]; //
Premier segment de l'URL
  const paramDynamique = segmentsActuels[1] || null; //Ex: id, nom, etc.

  const vueActuelle = this.routes[routeActuelle];
  if (vueActuelle) {
    const vueInstance = new vueActuelle(this.application, paramDynamique,
paramRecherche);
    vueInstance.afficher();
  } else {
    new Page404Vue(this.application).afficher();
  }
}
```

Gérer les vues

Une fois la route correspondante trouvée, on peut instancier la vue associée et l'afficher dans le conteneur principal de l'application. Chaque vue devrait avoir une méthode `afficher` qui gère le rendu du contenu. Certains frameworks comme React, Vue.js ou Angular fournissent des mécanismes intégrés pour gérer les vues et le DOM de manière efficace mais la logique reste similaire.

Les paramètres dynamiques vont souvent être utiles pour récupérer des données spécifiques à afficher dans la vue. Ex: La pizza avec l'ID 42 : `/produits/42`. Les paramètres de recherche peuvent également être utilisés pour filtrer ou trier les données affichées. Ex: `/produits?type=vegetarienne&tri=prix_asc`.

```
class AccueilVue {
  constructor(application, paramDynamique = null, paramRecherche = null) {
    this.application = application;
    this.paramDynamique = paramDynamique;
    this.paramRecherche = paramRecherche;
  }
}
```

```

    afficher() {
        //Rechercher les données nécessaires si besoin
        //Rendre le gabarit HTML dans le conteneur principal

        const gabarit = "<h1>Bienvenue sur la page d'accueil</h1>";
        this.application.conteneurPrincipal.innerHTML = gabarit;
        // Ajouter les écouteurs d'événements si nécessaire
    }
}

```

Changer l'historique sans recharger la page

À chaque clic, changement d'état ou navigation programmée, on peut utiliser la méthode `history.pushState()` pour mettre à jour l'URL affichée dans la barre d'adresse sans recharger la page.

La méthode `history.pushState()` prend trois arguments : un état (objet), un titre (string) et l'URL (string).

- L'état peut être utilisé pour stocker des informations supplémentaires sur l'état de la page.
- Le titre est généralement ignoré par la plupart des navigateurs. On le met souvent à une chaîne vide.
- L'URL est la nouvelle URL à afficher dans la barre d'adresse.

```

constructor() {
    window.addEventListener("popstate", this.onChangeementHistorique.bind(this));
    document.body.addEventListener("click", this.onClicLien.bind(this));
    this.routes = {
        "/": AccueilVue,
        "/produits": PageProduitVue,
        "/produits/:id": PageProduitDetailVue,
        // Ajouter d'autres routes ici
    };

    this.miseAJour(); // Initialiser la vue en fonction de l'URL actuelle
}

```

```

naviguerVers(chemin, recherche = "") {
    const urlComplete = chemin + recherche;
    history.pushState({}, "", urlComplete);
    this.miseAJour(); // Met à jour la vue en fonction de la nouvelle URL
}

```

Exemple complet de la classe Routeur

```

import AccueilVue from "../views/AccueilVue.js";
import PageProduitVue from "../views/PageProduitVue.js";
import PageProduitDetailVue from "../views/PageProduitDetailVue.js";

```

```

class Routeur {
  constructor(application) {
    this.application = application;
    this.routes = {
      "/": AccueilVue,
      "/produits": PageProduitVue,
      "/produits/:id": PageProduitDetailVue,
    };
    window.addEventListener("popstate",
this.onChangeementHistorique.bind(this));
    document.body.addEventListener("click", this.onClicLien.bind(this));
  }

  onChangeementHistorique(evenement) {
    this.miseAJour();
  }

  onClicLien(evenement) {
    if (evenement.closest("[data-lien]")) {
      evenement.preventDefault();
      const url = new URL(evenement.target.href);
      this.naviguerVers(url.pathname, url.search); // Fonction personnalisée
pour gérer la navigation
    }
  }

  naviguerVers(chemin, recherche = "") {
    const urlComplete = chemin + recherche;
    history.pushState({}, "", urlComplete);
    this.miseAJour(); // Met à jour la vue en fonction de la nouvelle URL
  }

  miseAJour() {
    const url = new URL(window.location.href);
    const cheminActuel = url.pathname;
    const paramRecherche = url.search;

    const segmentsActuels = cheminActuel.split("/");
    const routeActuelle = segmentsActuels[0] === "" ? "/" :
segmentsActuels[0]; // Premier segment de l'URL
    const paramDynamique = segmentsActuels[1] || null; //Ex: id, nom, etc.

    const vueActuelle = this.routes[routeActuelle];
    if (vueActuelle) {
      const vueInstance = new vueActuelle(this.application, paramDynamique,
paramRecherche);
      vueInstance.afficher();
    } else {
      new Page404Vue(this.application).afficher();
    }
  }
}

```